

Final Project

Graduate Course in Mechatronic Systems Nonlinear Control and Real-Time Digital Implementation

Shai Arogeti

Project objective. Select one nonlinear mechatronic process and develop two controllers for the same process: **(A)** an adaptive RBF neural-network controller for trajectory tracking, with online RBF-weight adaptation; and **(B)** a nonlinear optimal controller designed from data by off-policy adaptive dynamic programming (ADP).

Work stages. Parts 1–3 shall be developed and validated in MATLAB/Simulink or Python. They include process modeling, development of the adaptive RBF-NN controller, and development of the nonlinear off-policy ADP controller. Part 4 is dedicated to the digital and real-time implementation of the developed controllers on the Raspberry Pi Pico.

The project may be performed individually or in pairs. No preliminary proposal is required. Students who are uncertain about the suitability of a selected process may consult the instructor, but responsibility for selecting a process that satisfies the requirements remains with the students. Students may continue with the nonlinear process used in their homework assignment. When necessary, the model may be reformulated or adapted to the state-space representation and controller-development framework used in this project.

1. Selection and Modeling of the Process

The selected process shall be a continuous-time nonlinear system of order $n \geq 3$ represented in control-affine form,

$$\dot{x} = f(x) + g(x)u, \quad x \in \mathbb{R}^n, \quad u \in \mathbb{R}. \quad (1)$$

The complete process model shall be available for simulation and validation. In both controller designs, $g(x)$ is assumed known to the controller, whereas $f(x)$ is treated as an unknown smooth nonlinear function.

It shall satisfy the following requirements:

- 1.1** Every selected state shall have a physical meaning and participate in the control problem. Adding an unused, decoupled, or artificial state does not satisfy the order requirement.
- 1.2** The model shall represent a physical or mechatronic process and shall be supported by a scientific source, a textbook, or a documented first-principles derivation.
- 1.3** The process shall be significantly nonlinear over the operating range considered in the project. A nearly linear model with only a weak nonlinear correction is not sufficient.
- 1.4** Compare the nonlinear model with its local linearization in at least two representative cases within the selected operating range. Explain the observed differences and identify the nonlinear terms responsible for them.
- 1.5** Express the model so that the equilibrium used for regulation is located at the origin. The model shall satisfy $f(0) = 0$. If the physical equilibrium is $(x_e, u_e) \neq (0, 0)$, define suitable state and input deviation variables before developing the controllers.
- 1.6** For the zero-input system

$$\dot{x} = f(x), \quad (2)$$

provide a Lyapunov-based proof that the origin is at least locally asymptotically stable. Present the selected Lyapunov function, verify the required definiteness conditions, and state the region in which the result is valid.

- 1.7** Assume that the complete state vector is available to both controllers. Define the states, control input, operating range, input limits, and model parameters.

The same process shall be used in both project branches, although the tracking and regulation objectives may differ.

2. Branch A: Adaptive RBF-NN Tracking Control

This part shall be developed and validated in MATLAB/Simulink or Python. Formulate a trajectory-tracking problem for the selected process. The desired trajectory and any of its time derivatives used by the controller shall be specified explicitly. The selected trajectory must be sufficiently smooth and feasible for the chosen process.

- A1. Nominal continuous-time controller.** Develop a tracking controller for the nominal case in which $f(x)$ is assumed to be known exactly. Define the tracking errors and the nominal closed-loop error dynamics.
- A2. RBF approximation of the unknown nonlinear function.** Approximate the unknown nonlinear function using an RBF neural network, for example,

$$f(x) = W^{*T}\phi(x) + \varepsilon(x), \quad \hat{f}(x) = \widehat{W}^T\phi(x). \quad (3)$$

Justify the network inputs, approximation domain, centers, widths, and number of basis functions.

- A3. Adaptive controller and stability analysis.** Develop the adaptive control law and the RBF-weight adaptation law for $\widehat{W}$ using a Lyapunov analysis. Account explicitly for the RBF approximation error, state the assumptions used in the analysis, and state clearly the obtained stability, boundedness, and tracking result.
- A4. Discrete-time realization.** Based on the continuous-time adaptive controller developed above, use the indirect digital-design approach presented in the course to derive and implement a discrete-time realization of both the control law and the RBF-weight adaptation law. State the discretization method, calculation order, selected sampling period, and any limits imposed on the weights or control input.
- A5. Computer-based validation and comparison.** Use the same nonlinear process, reference trajectory, and initial conditions in all comparisons. First, compare the continuous-time adaptive RBF-NN controller with the corresponding continuous-time nominal controller. Present the time histories of the states, tracking error, control input, RBF approximation output, and representative weight estimates.

Next, compare the continuous-time adaptive controller with its discrete-time realization for at least three sampling periods, including one that produces a clear degradation in performance. Discuss the effects of sampling and numerical implementation.

3. Branch B: Nonlinear Off-Policy ADP

This part shall be developed and validated in MATLAB/Simulink or Python. The baseline task is optimal regulation of the selected nonlinear process.

- B1. Optimal-regulation problem.** Define the quadratic performance index

$$J(x_0, u) = \int_0^\infty \left[x^T(t)Qx(t) + u^T(t)Ru(t) \right] dt, \quad Q = Q^T > 0, \quad R = R^T > 0, \quad (4)$$

and explain how the selected weighting matrices represent state regulation and control effort.

- B2. Initial policy.** Use

$$u_0(x) = 0 \quad (5)$$

as the initial policy. Relate its admissibility to the zero-input stability result established in Part 1 and to the selected operating region and performance index.

- B3. Data generation.** Use a bounded and sufficiently exciting input signal to generate informative state and input data for the off-policy calculation. State the selected sampling period, data length, and integration windows, and verify that the state and input remain within the declared operating range.
- B4. Value-function approximation and policy improvement.** In the linear case, the value function has the form $V_i(x) = x^T P_i x$. For the selected nonlinear process, use the following value-function approximation and policy-improvement expression:

$$\hat{V}_i(x) = x^T P_i x + c_i^T \phi_{V, \text{nl}}(x), \quad (6)$$

$$\hat{u}_{i+1}(x) = -\frac{1}{2} R^{-1} g^T(x) \nabla \hat{V}_i(x). \quad (7)$$

Here, P_i is represented symmetrically and the selected basis functions are differentiable. The functions in $\phi_{V, \text{nl}}(x)$ shall represent nonlinear corrections to the local quadratic value function and shall vanish at the origin together with their first derivatives. For a polynomial approximation, use terms of total degree three or higher. Do not include constant or linear terms in the nonlinear value-function correction.

Select a limited set of differentiable nonlinear basis functions that reflects the nonlinear structure of the process and the selected operating region. Explain and justify the selected basis functions.

- B5. Relation to the policy approximation presented in the course.** Explain why the policy formulation used in this project differs from the separate policy approximation presented in the course slides.
- B6. Off-policy ADP equations and numerical solution.** Develop the off-policy equations used for policy evaluation and policy improvement and identify the unknown value-function coefficients in (6). State the dimensions of the resulting linear system and use enough data windows to obtain more equations than unknown coefficients. Calculate the coefficients using the pseudoinverse, state the stopping criterion, and reuse the same dataset throughout the policy iterations. After obtaining the matrices P_i , examine their eigenvalues and discuss whether the quadratic part of the approximated value function is locally positive definite.
- B7. Validation of the learned policy.** Compare the initial and final policies for several initial conditions within the selected operating region. Present the time histories of all state variables and the control input. In addition, calculate and compare the accumulated finite-horizon approximation

$$J_T = \int_0^T \left[x^T(t) Q x(t) + u^T(t) R u(t) \right] dt. \quad (8)$$

Discuss the results in relation to the selected weighting matrices and the resulting balance between state regulation and control effort. Report the convergence of the policy-iteration procedure, including the number of iterations, stopping criterion, and evolution of the identified value-function coefficients, resulting policy, and accumulated cost.

- B8. Extension to a tracking problem.** Propose a theoretically consistent extension of the developed nonlinear ADP method to a tracking problem for the same process. Define the reference signal, tracking error, modified formulation, and appropriate performance index. Explain the required changes in the ADP procedure and specify the additional information or data required by the proposed approach.

Because the process model is not fully known, a feedforward input that guarantees exact tracking may not be directly available. Exact asymptotic tracking is therefore not required unless it follows from clearly stated assumptions. Explain the expected behavior of the tracking

error, such as convergence to zero or to a bounded neighborhood of zero, and discuss the assumptions and limitations of the proposed formulation. The required deliverable for this item is the conceptual and mathematical formulation only.

4. Digital and Real-Time Implementation on the Raspberry Pi Pico

After completing Parts 1–3, implement both developed controllers on the provided Raspberry Pi Pico using MicroPython. No external physical plant is used. The nonlinear process shall therefore be simulated on the same Pico.

The Pico contains two processor cores. Both shall be used with the following fixed allocation:

- the **main/default core** shall execute the active controller;
- the **secondary core** shall simulate the complete nonlinear process.

At each controller update, the secondary core shall provide the complete simulated state vector to the main core, and the main core shall calculate and return the corresponding control input.

- C1. Two-core software architecture.** Present a block diagram and describe the functions executed by each core and the variables exchanged between them. The secondary core shall implement the complete nonlinear model. The main core may use the complete state vector and the known function $g(x)$. The unknown function $f(x)$ shall remain part of the process simulation and shall not be evaluated directly by the controller code.
- C2. Continuous-time process simulation and discrete-time controller timing.** The plant model represents a continuous-time nonlinear system and shall be simulated numerically approximately in real time using a fixed integration step dt . On the secondary core, configure a hardware timer with period dt . Each timer interrupt shall trigger one numerical integration step of the process model. Select dt sufficiently small relative to the process dynamics and state the numerical integration method used.

The controller shall be implemented as a discrete-time controller with sampling period T_s . Its update rate shall be lower than the process-simulation rate, with

$$T_s = Ndt, \quad N > 1, \quad (9)$$

where N is an integer. On the main core, configure a hardware timer with period T_s . Each timer interrupt shall trigger one controller update using the most recent complete state vector. Hold the control input constant between controller updates.

- C3. Adaptive RBF-NN implementation.** Implement the adaptive controller on the main core. At each controller update, read the complete state vector, calculate the control input, update the RBF weights, and transfer the new control input to the secondary core. Compare the Pico responses with the corresponding MATLAB/Simulink or Python results under matching conditions.
- C4. ADP final-policy implementation.** Implement on the main core the final ADP policy obtained and validated in Part 3. Transfer the identified value-function coefficients and selected basis functions to the controller code. At each controller update, evaluate the gradient of the final value-function approximation and calculate the control input using (7). The identified coefficients shall remain fixed during the Pico implementation. Compare the Pico responses with the corresponding computer-based results.
- C5. Numerical operations and execution-time measurement.** The Pico implementation shall use `uLab` for the vector and matrix operations required by the controllers. Whenever applicable, use `uLab` array operations rather than element-by-element Python loops.

Measure the execution time of each controller update on the main core. The measured interval shall include reading the current state vector, calculating the control input, updating the

RBF-NN weights when applicable, and transferring the control input to the secondary core. Report the average and maximum execution times and verify that the maximum controller execution time is smaller than the selected sampling period.

5. Report Submission and Oral Defense

5.1. Required report structure

The submitted report shall follow the section structure specified below. The principal section titles and numbering shall be retained; additional subsections may be added when necessary.

1. Introduction and Project Overview

2. Nonlinear Process Model and Analysis

- 2.1. Physical process, model source, and modeling assumptions
- 2.2. Mathematical model, operating range, known input function $g(x)$, and unknown nonlinear function $f(x)$
- 2.3. Regulation equilibrium and Lyapunov stability of the zero-input system
- 2.4. Significance of the nonlinear dynamics

3. Adaptive RBF-NN Tracking Controller

- 3.1. Tracking objective and nominal continuous-time controller
- 3.2. RBF approximation, adaptive controller, weight-adaptation law, and stability analysis
- 3.3. Discrete-time realization
- 3.4. Computer-based validation and continuous/discrete comparison

4. Nonlinear Off-Policy ADP

- 4.1. Regulation problem, performance index, and initial policy
- 4.2. Data generation
- 4.3. Value-function approximation and policy improvement
- 4.4. Relation to the policy approximation presented in the course
- 4.5. Off-policy equations, numerical solution, policy iterations, and validation
- 4.6. Proposed extension to a tracking problem

5. Raspberry Pi Pico Implementation

- 5.1. Two-core architecture and real-time process simulation
- 5.2. Adaptive RBF-NN implementation
- 5.3. ADP final-policy implementation
- 5.4. Execution-time measurement and comparison with computer results

6. Conclusions and Limitations

Where relevant, the report shall include a reference list for external sources used in selecting the process model or supplementing the course material. Appendices may be used for long code listings, additional tables, or supporting derivations that are not required in the main discussion.

5.2. Submitted material

Submit one PDF report and an organized archive containing all MATLAB/Simulink or Python files, MicroPython source code, parameters, and data required to reproduce the presented results. Equations, figures, tables, and algorithms shall be numbered and referenced. The code shall be organized and documented sufficiently to identify the process simulation, controller implementations, numerical procedures, and communication between the two Pico cores.

5.3. Group oral defense and live demonstration

A single defense session will be scheduled for each project group after submission. All group members shall attend and participate together. During the defense, the students shall present and explain their work directly from the submitted report, demonstrate the Pico implementation, and answer technical questions concerning the mathematical derivations, simulations, code, implementation decisions, and obtained results. No separate presentation is required.

During the oral defense, questions may extend beyond the specific derivations presented in the report to related concepts and methods covered in the course. Students are expected to be familiar with the full course material and to explain how the relevant principles apply to their project.

Participation in the defense is mandatory for every student. A student who does not participate in the defense will not receive a project grade. Each student may be questioned about any part of the project.

6. Assessment

The final project grade is divided into four written and implementation components of 15 points each and an oral-defense component of 40 points. The written components are common to the project group. The oral-defense grade is assigned individually according to the knowledge and understanding demonstrated by each student.

Code	Nonlinear process modeling and analysis	Points
M1	Suitability of the selected nonlinear process, model source, assumptions, variables, and operating region	4
M2	Correct control-affine formulation and clear definition of the known function $g(x)$ and unknown function $f(x)$	3
M3	Origin-centered equilibrium and Lyapunov analysis of the zero-input system	4
M4	Demonstration and discussion of the significance of the nonlinear dynamics relative to the local linearization	4
Part 1 total		15
Code	Adaptive RBF-NN tracking controller	Points
A1	Tracking-problem formulation and derivation of the nominal continuous-time controller	3
A2	RBF-NN approximation and derivation of the adaptive controller and weight-adaptation law	4
A3	Lyapunov-based stability or boundedness analysis, including the approximation error	4
A4	Continuous-time results, discrete-time realization, and required comparisons	4
Part 2 total		15

Code	Nonlinear off-policy ADP controller	Points
B1	Optimal-regulation formulation, selection of Q and R , and data-generation procedure	3
B2	Selection and justification of the value-function approximation, understanding of the gradient-based policy-improvement formulation, and explanation of its difference from the separate policy approximation presented in the course slides	4
B3	Derivation and pseudoinverse-based numerical implementation of the off-policy equations	4
B4	Policy-iteration results, validation of the learned policy, accumulated-cost comparison, and tracking-extension formulation	4
Part 3 total		15
Code	Raspberry Pi Pico implementation	Points
P1	Two-core architecture and approximately real-time simulation of the continuous-time nonlinear process	3
P2	Digital implementation of the adaptive controller, including the RBF-weight updates	3
P3	Digital implementation of the final ADP policy from the identified value-function approximation	3
P4	Use of <code>ulab</code> , exchange of the state and control input between the cores, and organization of the MicroPython code	2
P5	Controller execution-time measurement and comparison of the Pico and computer-based results	4
Part 4 total		15
Code	Oral defense and live demonstration - individual assessment	Points
V1	Presentation and explanation of the submitted report, results, and Pico implementation	15
V2	Responses to questions about the project, including its model, derivations, controllers, simulations, code, results, and implementation	15
V3	Demonstrated knowledge of the course material and ability to relate relevant concepts and methods to the project	10
Oral-defense total		40